Arreglos (Arrays)

- Estructuras de datos
- Tipos de estructuras.
  - Internas
  - Externas
- Dimensionar
- ¿Como crear un arreglo?
- Acceso secuencial
- Acceso directo
- Ejemplos
- **Arreglos de 2 Dimensiones**
- definición
- Recorridos por Fila / Columna

ARREGLOS (VECTORES)

INTRODUCCION. ESTRUCTURAS DE DATOS.

Hasta ahora, para hacer referencia a un dato usábamos una variable.

A = 4 **A** | 4 |

El problema se plantea cuando tenemos **gran cantidad de datos que guardan entre sí una relación.**

Ejemplo: realizar un programa que permita ingresar los datos **de todas las facturas del año** pasado y obtener el **total mensual**. Por cada factura se ingresará el número de mes y el importe total.

¿Cuántos totales necesitamos?

Total-enero, total-febrero, ..., total-diciembre

Para cada uno de estos datos se debería utilizar una variable distinta, esto acarrea una gran laboriosidad la hora de construir el programa, unida a la cantidad de variables a usar.

Para resolver esto, se **agrupan datos en un mismo conjunto, bajo un nombre común**, que se **puede tratar como una sola unidad**.

Estos conjuntos reciben el nombre de

estructuras de datos.

Tipos de estructuras.

dependiendo del lugar donde residen o se almacenen, se clasifican en:

- ***internas***: residen en la **memoria del ordenador**, ejemplo: ***vectores*** *(arreglos o arrays)*
- ***externas***: residen en un **soporte externo**, ejemplo: ficheros o archivos que se almacenan en dispositivos magnéticos.

ESTRUCTURAS INTERNAS DE DATOS.

Un ***vector*** (*arreglos o arrays*) es una **estructura interna** de datos con ***un conjunto de elementos*** homogéneos ***del mismo tipo*** (*numérico, alfanumérico o booleanos*) reconocido por un ***nombre en común***, que reside en la memoria del ordenador.

A cada ***elemento*** ***se accede por la posición*** que ocupa dentro del conjunto de datos.

Según el tipo de datos almacenados en el vector, estos pueden ser:

- ***numérico***
- ***cadenas de caracteres***
- ***booleanos***
- ***etc.***

Antes de poder utilizarlos:

- hay que **reservar una zona de la memoria para su uso**

- **definir el número de parámetros** necesario para acceder a cada elemento de la estructura: **dimensionarlos**.

Según el número de parámetros necesarios para dimensionar los arrays, se pueden clasificar en los siguientes tipos:

- Unidimensionales.
- Bidimensionales.
- Multidimensionales.

VECTORES o ARREGLOS unidimensionales

También se llaman ***arrays*** *unidimensionales o* simplemente ***listas***.

Es un conjunto de ***n*** datos homogéneos, cada uno de los cuales debe ser referenciado por un **índice**, **I**, que debe cumplir con la siguiente propiedad:

(I >= 0) y (I <= n)

Al proceso de **reservar una zona** de la memoria para almacenar los datos del vector se llama ***dimensionar***.

¿Cómo creamos un arreglo?

Para crear un arreglo se debe definir:

1° - el **tipo** y el **nombre**, como cualquier variable

2° - **dimensionamos:** cantidad de elementos.

Ejemplo:

1° Definir la variable como arreglo: en el ejemplo vamos a definir un arreglo **nombre** del tipo **caracter** (para guardar datos alfanuméricos)

Definir *cant, i* ***Como Entero;***

Definir *nombre* ***Como caracter;***

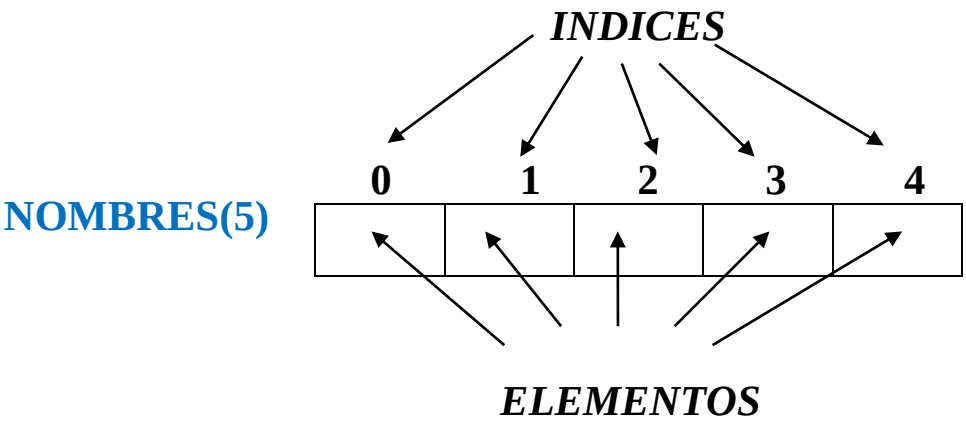
2° Dimensional el arreglo (definir cantidad de elementos)

Dimension *nombre[5];*

El nombre de un arreglo debe cumplir con las mismas normativas que el nombre de una variable.

Cuando *dimensionamos* un **ARREGLO**, estamos creando la **estructura** (cantidad máxima de elementos) y reservando el lugar en la memoria.

Gráficamente, podríamos representarlo de la siguiente manera:



Ahora cada vez que queremos hacer referencia a un ***elemento*** del vector, debemos indicarlo a través del ***índice***.

NOMBRE-VECTOR[INDICE]

DEFINICION DE LOS ELEMENTOS (Tipos de datos)

Todos los elementos de un vector **van a ser del mismo tipo** (numéricos, alfanuméricos o booleanos).

Ejemplo: si quisiera cargar el vector ***NOMBRE(5)*** con nombres de personas, primero debo definirlo como alfanuméricos.

```
Algoritmo ejempo_carga_arreglos
    Definir   cant,  i Como Enteros;
    Definir nombre, nombre_socio Como Caracter;
    Dimension nombre[5];

    Para i = 0  Hasta 4 Con Paso 1 Hacer
        Escribir "ingrese nombre del socio"
        Leer nombre_socio
        nombre[i]=nombre_ socio;
    Fin Para
FinAlgoritmo
```

Una vez ejecutado el programa quedara en memoria algo así:

0	1	2	3	4
JUAN	LUIS	PEDRO	MARIA	LAURA

de manera tal que:

NOMBRE [**0**] = "JUAN"
NOMBRE [**1**] = "LUIS"
NOMBRE [**2**] = "PEDRO"
NOMBRE [**3**] = "MARIA"
NOMBRE [**4**] = "LAURA"

- Cada *elemento* de un ***vector***, puede cumplir la mismas funciones que una variable: contador, acumulador, señal o simplemente para guardar un dato.

- Si los utilizamos como **acumuladores** o **contadores**, deben inicializarse con ceros, al igual que las variables que cumplen con dicha función.

Sea por ejemplo el algoritmo necesario para inicializar con ceros el vector **CUENTA(5)**.

Necesitamos una estructura repetitiva, que nos permita recorrer los elementos del vector en forma secuencial.

Para ello los valores que debe tomar el índice serán 0 a 4.

Si observamos los valores de este, vemos que son los mismos que toma una variable contadora, que **comienza con 0, finaliza con 4 y se incrementa de 1 en 1.**

Por lo tanto, **para recorrer secuencialmente los elementos** necesitamos una estructura repetitiva exacta, con 5 ocurrencias (una por elemento), donde la variable contadora, que controla dicho ciclo también será utilizada como índice de los elementos del vector.

Permite recorre los elementos del vector (de 0 a 4)

CUENTA(5)

0	1	2	3	4

Algoritmo **inicializa_arreglo_en_cero**

```
//1° Definir la variable: definimos el arreglo  con la variable cuenta
Definir  i  Como Enteros;
Definir  cuenta  Como Real;
//2° Dimensional el arreglo (definer cantidad de elementos)
Dimension cuenta[5];
Para i = 0  Hasta 4 Con Paso 1 Hacer
        cuenta[i]=0 ;
Fin Para
FinAlgoritmo
```

cuenta[5]

0	1	2	3	4

Una vez finalizado el programa el vector ***cuenta(5)*** quedará cargado de la siguiente manera:

cuenta[5]

0	1	2	3	4
0	0	0	0	0

Recorriendo el vector de esta manera (elemento por elemento) la estamos accediendo en forma ***secuencial***

ACCESO DIRECTO.

Los elementos de un ***vector*** son también de **acceso directo**.

Esto significa que, **para acceder a un elemento en particular**, si conocemos la posición, **no necesariamente tendremos que pasar por los elementos anteriores** a este.

Obviamente para esto **primero necesitamos conocer la ubicación** del **elemento *[índice],*** y luego, **si es correcto,** podremos acceder al mismo.

IMPORTANTE: *siempre que haga* ***acceso directo*** *a un elemento del vector, debo* ***validar*** *que* ***el valor del índice sea correcto****.*

Significa que sea mayor o igual que cero y menor o igual que el definido en el dimensionamiento

Ejemplo en un arreglo **numero*[5],*** quiero mostrar por pantalla el valor del **3° elemento** (índice 2)

	0	1	2	3	4
numero[5]	**0**	**0**	**0**	**0**	**0**

Escribir *numero*[2]

Ejemplo: definido y cargado el vector ***edades [4],*** realizar el pseudocódigo de un programa que permita ingresar la posición del elemento que quiera ver, y mostrarle el contenido.

```
Algoritmo acceso_directo
    Definir  i, x Como Enteros;
    Definir cuenta  Como real;

    Dimension  cuenta[5];

    Para i = 0 Hasta 4 Con Paso 1 Hacer
        cuenta[i]=0 ;
    Fin Para

    Escribir "Ingrese el N° del elemento que quiere consultar: "
    Leer x

    // validación del índice
    si (x >= 0) y ( x <= 4)
        Escribir "El valor del elemento N° " x " es " cuenta[x]
    sino
        Escribir "El número de elemento ingresado es erróneo"
    FinSi
FinAlgoritmo
```

Modificación del algoritmo para **validar el dato**, de manera tal que hasta que no lo ingrese correctamente no continúe.

```
Algoritmo acceso_directo
        Definir   i, x Como Enteros;
        Definir cuenta   Como real;
        Dimension cuenta[5];

    // carga el arreglo con datos desde el teclado
        Para i = 0 Hasta 4 Con Paso 1 Hacer
                Escribir "Ingrese un valor numérico "
                Leer cuenta[i]
        Fin Para

     // ingresa el valor de la posición que quiere acceder
        Escribir "Ingrese el N° del elemento que quiere consultar: "
        Leer x

    // valida el valor ingresado, si es erróneo lo vuelve a pedir
        Mientras ((x < 0) o ( x > 4))
                Escribir "El n° es erróneo. Ingrese nuevamente "
                Leer x
        Fin Mientras

    // muestra el valor del elemento
        Escribir "El valor del elemento N° " x " es " cuenta[x];
FinAlgoritmo
```

ARREGLOS de 2 DIMENSIONES

• Definición: carga y visualizacion

```
Algoritmo ejempo_arreglos_2d
    //1° Definir la variable: definimos el arreglo  con la variable NOMBRE
    Definir   cant,  f, c Como Enteros;
    Definir nombre, nombre_socio  Como Caracter;

    //2° Dimensional el arreglo (definer cantidad de filas y columnas)
    Dimension nombre[4,3];

    // Carga el arreglo con datos desde el teclado
    Para f = 0  Hasta 3 Con Paso 1 Hacer
        Para c = 0  Hasta 2 Con Paso 1 Hacer
            Escribir "ingrese nombre del articulo"
            Leer nombre_socio
            nombre[f,c]=nombre_socio ;
        Fin Para
    Fin Para

    // Mostrar datos del arreglo por pantalla
    Para f = 0  Hasta 3 Con Paso 1 Hacer
        Para c = 0  Hasta 2 Con Paso 1 Hacer
            Escribir nombre[f,c];
        Fin Para
    Fin Para
FinAlgoritmo
```

Recorridos por Fila / Columna

FILAS COLUMNAS

	0	1	2	3	4
0	(0, 0)	(0, 1)	(0, 2)	(0, 3)	(0, 4)
1	(1, 0)	(1, 1)	(1, 2)	(1, 3)	(1, 4)
2	(2, 0)	(2, 1)	(2, 2)	(2, 3)	(2, 4)
3	(3, 0)	(3, 1)	(3, 2)	(3, 3)	(3, 4)

Supongamos que tenemos la matriz ***a[4][5]*** cargadas con nombres.

	0	1	2	3	4
0		JUAN			
1				LUIS	
2			PEDRO		
3	PEPE				

de manera tal que:

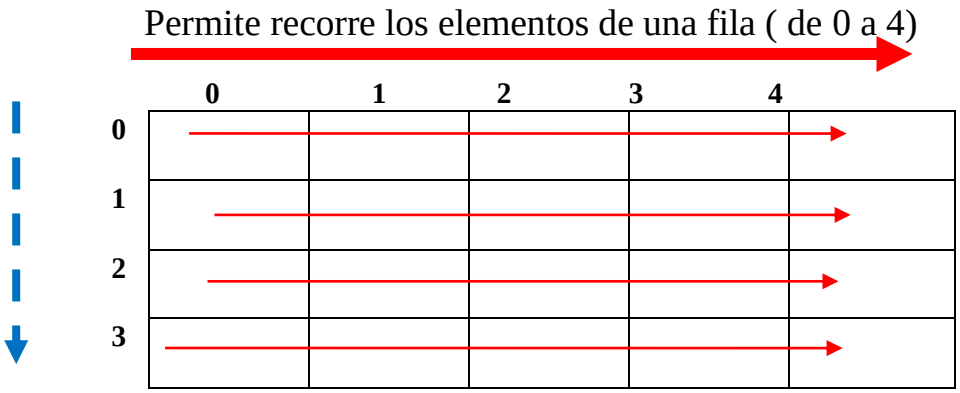
a[0][1] = "**JUAN**" a[1][3]= "LUIS"

a[2][2] = "PEDRO" a[3][0] = "PEPE"

Para inicializar los elementos de una matriz, al igual que las tablas debo hacerlo elemento por elemento, ya sea **por fila o por columnas.**

Sea por ejemplo el algoritmo necesario para inicializar con ceros la matriz ***a[4][5]*** recorriéndola por fila.

Para este algoritmo necesitamos dos estructuras repetitivas, una que nos permita recorrer los elementos de una filas (*columnas*) y otra que nos permita recorrer las filas.

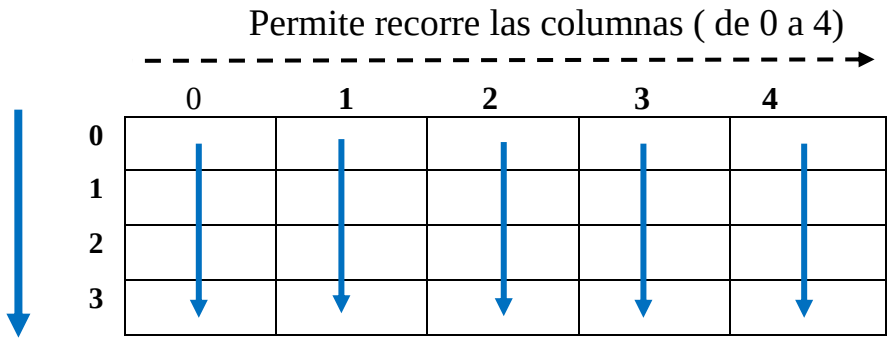


Por cada fila voy a tener que inicializar **5 elementos** (como si fuera un vector), y este proceso lo debo repetir para cada una de las 4 filas.

```
// Mostrar datos del arreglo por pantalla
Para f = 0  Hasta 3 Con Paso 1 Hacer
    Para c = 0  Hasta 4 Con Paso 1 Hacer
        Escribir nombre[f,c];
    Fin Para
Fin Para
```

Repetitiva por **Fila**

Repetitiva por **columna**



Repetitiva por **columna**

Para C = 0 Hasta 4 **Con Paso 1 Hacer**

Repetitiva por **Fila**

Para f = 0 Hasta 3 **Con Paso 1 Hacer**

Escribir nombre[**f**,**C**];

Fin Para

Fin Para

EJEMPLO: realizar un algoritmo de un programa que permita:

definir un arreglo de 2 dimensiones, de 2 filas por 3 columnas

- Cargar con datos numéricos por teclado
- Mostrar los datos del arreglo por pantallas, recorriéndolo por fila
- Mostrar los datos del arreglo por pantallas recorriéndolo por columna